SENTINEL

File Transfer Management System

Complete System Documentation

Version 1.0 | March 2026

Project	FTMS Sentinel – Security Monitoring System
Repository	github.com/Cybernextin/FTMS-Sentinal
Stack	Python | HTML | JavaScript | CSS
Status	**Production Ready | Defense Protocol: ACTIVE**

1. Introduction

1.1 Project Overview

Sentinel is an advanced, production-ready Secure File Transfer Monitoring System designed to oversee, track, and protect critical storage sectors in real time. It combines asynchronous threat detection, role-based access control, file manipulation forensics, and malware heuristics into a unified web-based platform.

1.2 Purpose & Goals

- Monitor a designated storage volume continuously for unauthorized file operations.
- Detect ransomware behavior, data ex-filtration, and suspicious modifications in real time.
- Provide administrators with a dashboard to view logs, manage users, and take defensive action.
- Generate detailed security reports for compliance and forensic investigation.
-

1.2 Key Definitions

Term	Definition
FTMS	File Transfer Management System
RBAC	Role-Based Access Control – restricts access based on assigned roles
Heuristic	Pattern-based threat detection without exact signatures
Nuclear Wipe	Irreversible tactical data purge via Sec-Ops mode
Quarantine	Isolated storage sector for flagged/suspicious files
Sector	Named storage partition within the monitored file-system root

2. System Architecture

2.1 Directory Structure

Directory / File	Description
server/	Flask backend: API routes, database_manager.py, HTML templates, JS components.
core/	Security backbone: monitor.py (watchdog), threat_engine.py, severity_engine.py, generate_report.py.
config/	Security parameters, network bindings, database config, and .env definitions.
storage/	Primary monitored file-system root containing .quarantine and user sectors.
logs/	Auto-generated JSON and text audit telemetry organized by date.
scripts/	Automation and testing scripts.
utils/	Shared utility functions across core and server.
run.py	Unified entry point – starts Flask server, watchdog daemon, and threat engine.
requirements.txt	All Python package dependencies.

2.2 Event Data Flow

File operation in storage/ → monitor.py detects event → threat_engine.py runs heuristic analysis → severity_engine.py assigns risk level → event logged to logs/ → Flask API exposes it to dashboard → generate_report.py compiles audit reports.

3. Installation & Setup

3.1 Prerequisites

Requirement	Details
Python	3.8 or higher (3.10+ recommended)
pip	Bundled with Python 3.4+
Git	For cloning the repository
OS	Linux, macOS, or Windows (WSL recommended)

3.2 Installation Steps

Clone & Enter Project

```
git clone https://github.com/Cybernextin/FTMS-Sentinal.git
cd FTMS-Sentinal
```

Create Virtual Environment

```
python -m venv venv
source venv/bin/activate      # Linux/macOS
venv\Scripts\activate         # Windows
```

Install Dependencies

```
pip install -r requirements.txt
```

Configure .env

```
IP_DETECTION_MODE=auto
IP_VERSION_PRIORITY=ipv4
AI_REPORT_API_KEY=your_api_key_here
```

Start the Application

```
python run.py
```

Dashboard accessible at http://localhost:5000 after startup.

3.3 Default Credentials

Field	Value
Username	admin
Password	password123

Security Notice: Change the default password immediately after first login.

4. Dashboard UI – Screen-by-Screen Guide

Every screen of Sentinel is documented below with an indexed element table. The # column identifies each UI element by number, matching the order elements appear on screen.

4.1 Administrator Login Portal

[SCREEN 1] Admin Login Portal

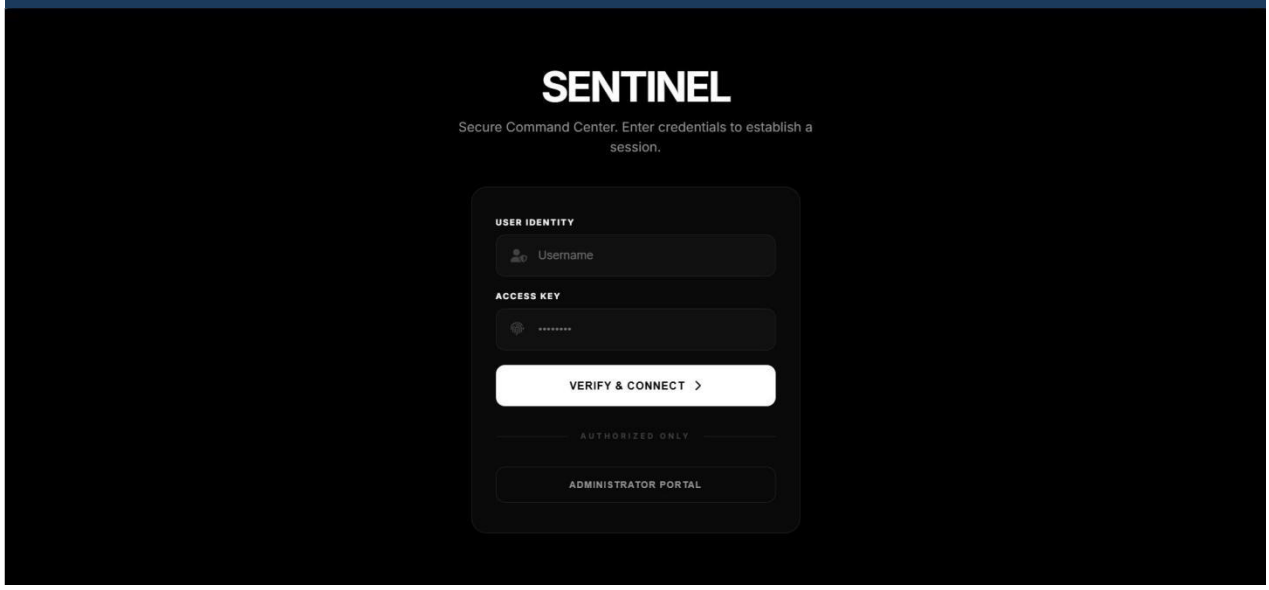

Fig 1 – Admin Login Portal: Secure Command Center entry for administrators.

No	UI Element	Description
1	**USER IDENTITY field**	Text input for the administrator username.
2	**ACCESS KEY field**	Masked password entry. Fingerprint icon signals secure input.
3	**VERIFY & CONNECT button**	Submits credentials to establish an authenticated admin session.
4	**AUTHORIZED ONLY banner**	Access-restriction notice displayed between the form and portal link.
5	**ADMINISTRATOR PORTAL link**	Routes directly to the admin-specific portal view.

4.2 User Login Page

[SCREEN 2] Command Center Access – User Login

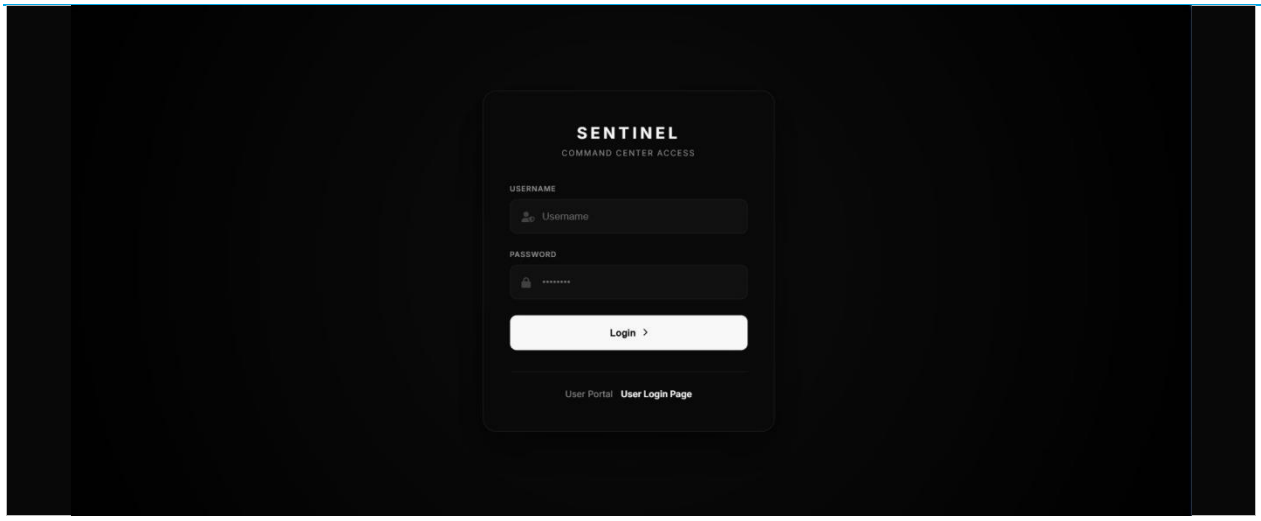

Fig 2 – User Login Page: Standard login for non-admin accounts.

No	UI Element	Description
1	**SENTINEL / COMMAND CENTER ACCESS header**	Identifies this as the standard user-facing login.
2	**USERNAME field**	Text input for the standard user's username.
3	**PASSWORD field**	Masked password entry with lock icon.
4	**Login button**	Authenticates the user and grants Workspace access.
5	**User Portal / User Login Page link**	Navigation shortcut confirming this is the user portal entry.

4.3 Live Monitor – Command Center Area

[SCREEN 3] Live Monitor Dashboard

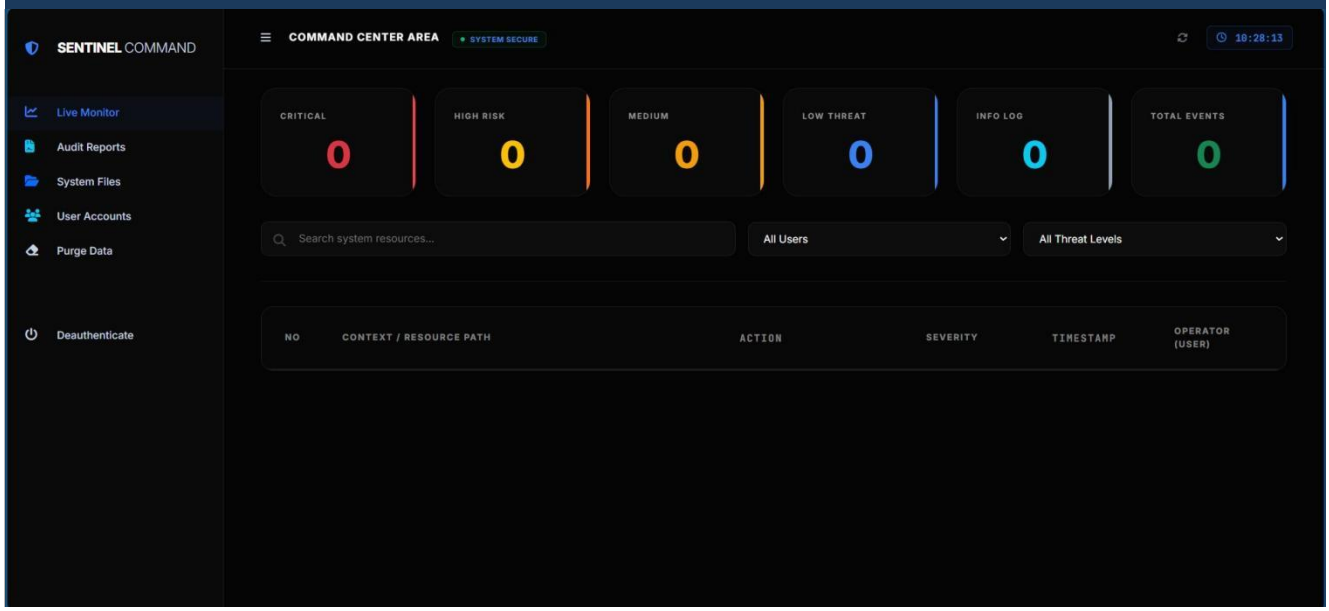

Fig 3 – Live Monitor: Real-time threat dashboard with severity counters and indexed event log table.

No	UI Element	Description
1	**CRITICAL counter (red)**	Count of CRITICAL severity events. Triggers immediate action.
2	**HIGH RISK counter (orange)**	Count of HIGH severity events. Requires prompt review.
3	**MEDIUM counter (yellow)**	Count of MEDIUM severity events. Investigate within standard SLA.
4	**LOW THREAT counter (blue)**	Count of LOW severity events. Informational; routine monitoring.
5	**INFO LOG counter (cyan)**	Non-threat informational events logged for audit completeness.
6	**TOTAL EVENTS counter (green)**	Cumulative count of all events across all severity levels.
7	**Search bar**	Full-text search across resource paths in the event log.
8	**All Users filter**	Drop-down to filter the log by a specific operator account.
9	**All Threat Levels filter**	Drop-down to filter the log by severity level.
10	**NO column (Index)**	Sequential index number auto-assigned to every logged event. Used as the primary reference key when generating audit reports by ID or numeric range (see Section 4.7). Each event's NO is permanent and unique within the log.
11	**CONTEXT / RESOURCE PATH column**	Full filesystem path of the affected resource.
12	**ACTION column**	The filesystem operation that triggered the event (CREATE, MODIFY, RENAME, DIRECTORY, OVERWRITE).
13	**SEVERITY column**	Color-coded severity rating assigned by severity_engine.py.
14	**TIMESTAMP column**	Exact date and time the watchdog daemon detected the event.
15	**OPERATOR (USER) column**	Authenticated user account associated with the file operation.
16	**SYSTEM SECURE badge**	Green top-bar badge indicating no active threats; turns red on detection.
17	**Live clock (top-right)**	Real-time server clock for timestamp correlation.

About the NO (Index) Column: The NO column assigns a permanent sequential number to every event as it is logged. When generating audit reports, operators use these numbers to select entries — either by individually checking rows (Selected Logs / IDs mode) or by specifying a start and end NO value (Range 1 to N mode). This makes cross-referencing between live logs and reports precise and reliable.

4.4 System Files – Admin File Explorer

[SCREEN 4] Admin File Explorer

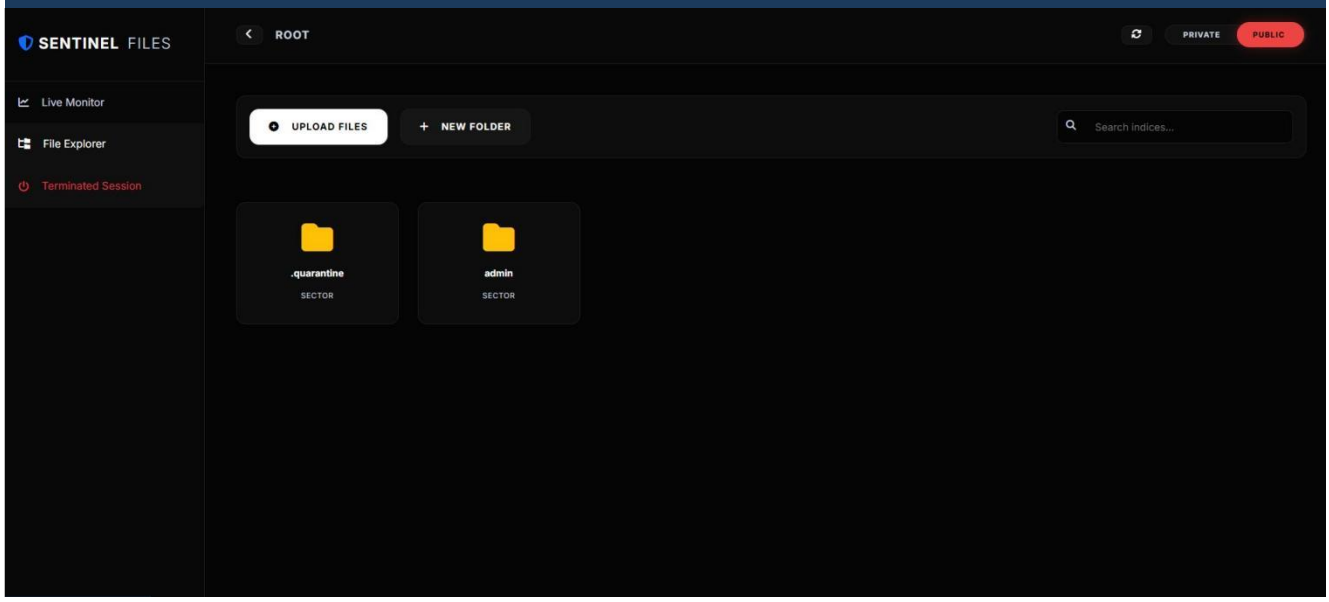

Fig 4 – Admin File Explorer: ROOT sector view with upload, folder management, and Private/Public toggle

No	UI Element	Description
1	**ROOT breadcrumb / back arrow**	Shows current directory path. Click back arrow to navigate up.
2	**PRIVATE / PUBLIC toggle**	Switches sector access mode. PUBLIC = visible to standard users; PRIVATE = admin-only.
3	**UPLOAD FILES button**	Opens a file picker to upload files into the current sector. All uploads are tracked by monitor.py.
4	**NEW FOLDER button**	Creates a new named sub-sector within the current directory.
5	**Search indices bar**	Searches file names and paths within the current sector.
6	**.quarantine sector**	System-reserved sector where the threat engine automatically isolates flagged files.
7	**admin sector**	Administrator-owned storage sector. Access-controlled and fully monitored.
8	**Terminated Session (sidebar)**	Immediately invalidates the current session — useful after emergency operations.

4.5 User Accounts Matrix

[SCREEN 5] User Accounts Matrix

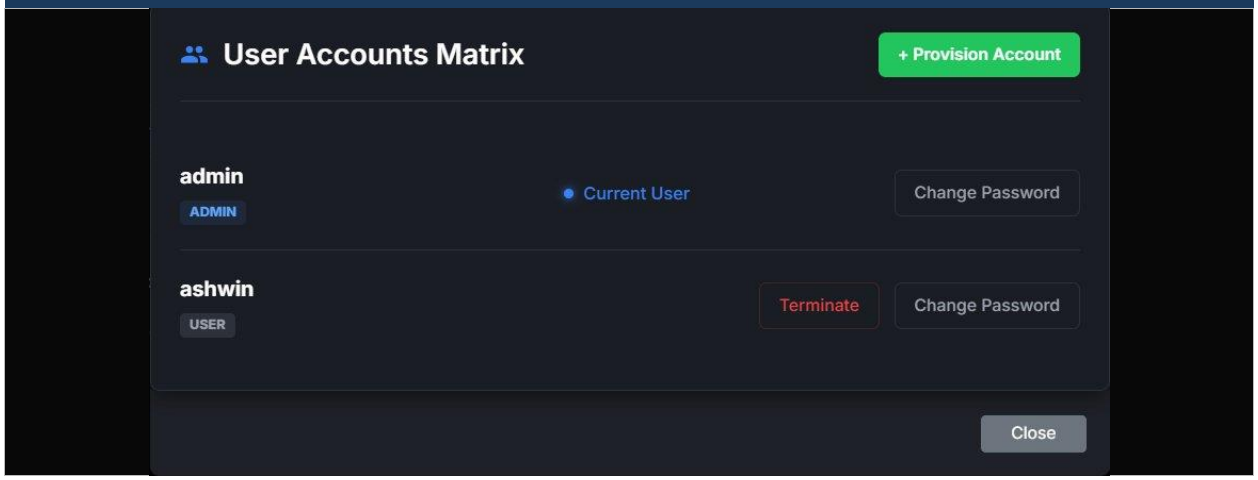

Fig 5 – User Accounts Matrix: RBAC user management panel with provisioning and account controls.

No	UI Element	Description
1	**+ Provision Account button**	Creates a new user account. Admin assigns username, password, and role.
2	**admin account row**	Currently logged-in admin. 'Current User' indicator shown. Only Change Password available (cannot self-terminate).
3	**ashwin account row**	Standard USER account. Shows role badge and offers Terminate and Change Password actions.
4	**ADMIN / USER role badges**	ADMIN = full system access. USER = Workspace-only access.
5	**Terminate button (red)**	Immediately deactivates the target user and invalidates all active sessions.
6	**Change Password button**	Opens a dialog to reset the selected user's password.

4.6 Log Purge Engine

[SCREEN 6] Log Purge Engine

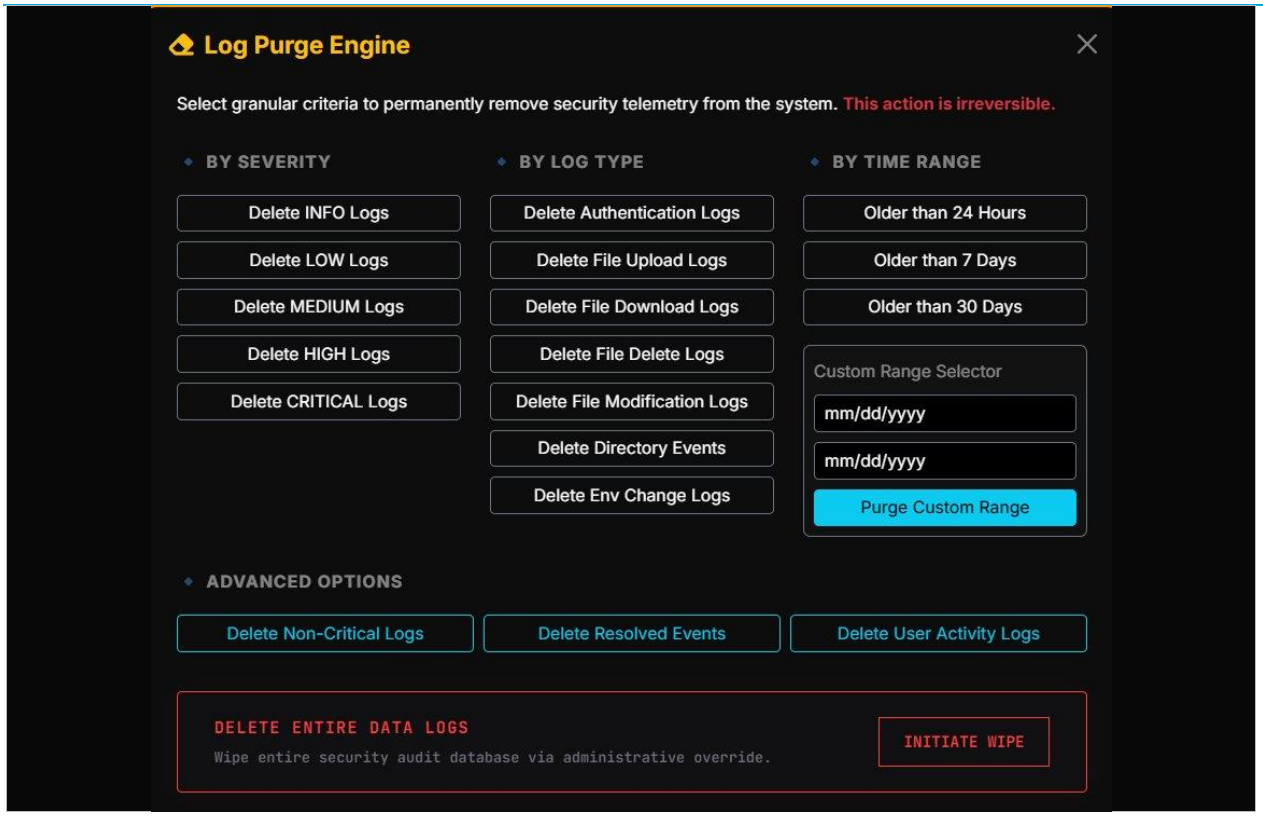

Fig 6 – Log Purge Engine: Granular log deletion by severity, type, time-range, and full-wipe options.

No	UI Element	Description
1	**BY SEVERITY section**	Five buttons to delete all logs at a specific severity: INFO, LOW, MEDIUM, HIGH, CRITICAL.
2	**BY LOG TYPE section**	Deletes logs by operation type: Authentication, Upload, Download, Delete, Modification, Directory Events, Env Change.
3	**BY TIME RANGE – presets**	Purge logs older than: 24 Hours / 7 Days / 30 Days.
4	**Custom Range Selector**	Date picker (mm/dd/yyyy) to define a custom purge date window.
5	**Purge Custom Range button**	Executes the purge for the custom date range defined above.
6	**Delete Non-Critical Logs**	Advanced option: removes all logs with severity below HIGH in a single action.
7	**Delete Resolved Events**	Advanced option: removes events that have been marked resolved.
8	**Delete User Activity Logs**	Advanced option: removes all user-action audit entries.
9	**DELETE ENTIRE DATA LOGS / INITIATE WIPE**	Nuclear Wipe: permanently destroys the entire security audit database. Requires Sec-Ops authorization. Completely irreversible.

Critical Warning: INITIATE WIPE is fully irreversible. Ensure all required forensic evidence and compliance backups are secured before activation.

4.7 Generate Audit Report

[SCREEN 7] Audit Report Generator

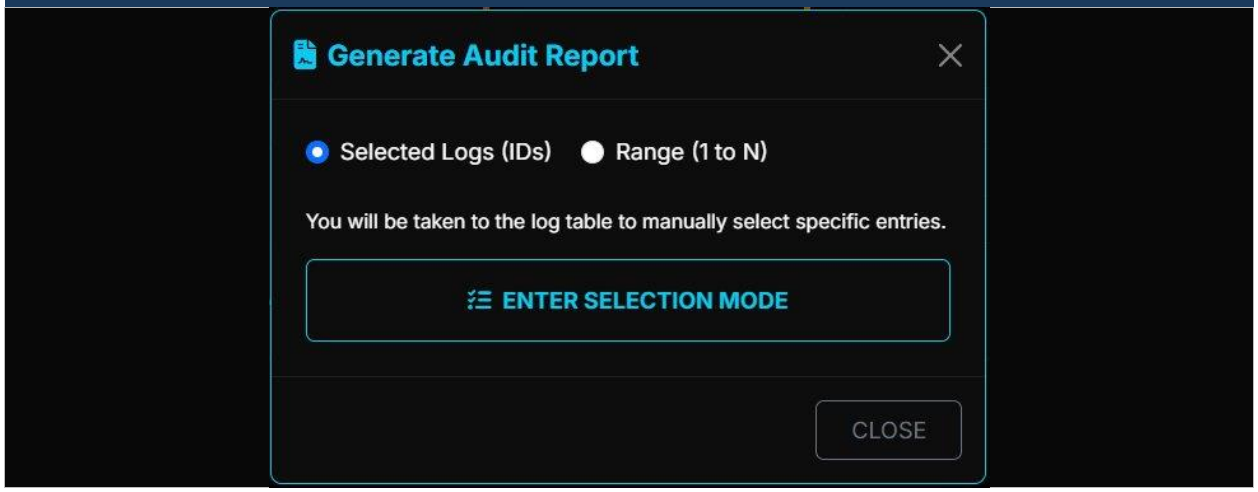

Fig 7 – Audit Report Generator: Modal for selecting log entries by index ID or numeric range.

No	UI Element	Description
1	**Selected Logs (IDs) mode**	Enters the live log table in selection mode. Operator checks specific rows by their NO index number to include in the report.
2	**Range (1 to N) mode**	Operator enters a start and end NO value. All events within that index range are automatically included in the report.
3	**ENTER SELECTION MODE button**	Navigates to the event log table with checkboxes enabled for manual per-row selection.

How the NO Index Powers Reports: In 'Selected Logs (IDs)' mode, you select entries by their NO number. In 'Range (1 to N)' mode, you enter a start and end NO. Both methods reference the NO column in the Live Monitor (Section 4.3, element #10), making that index column the backbone of all audit report generation.

4.8 Sentinel Workspace – User File Ecosystem

[SCREEN 8] User Workspace – File Ecosystem

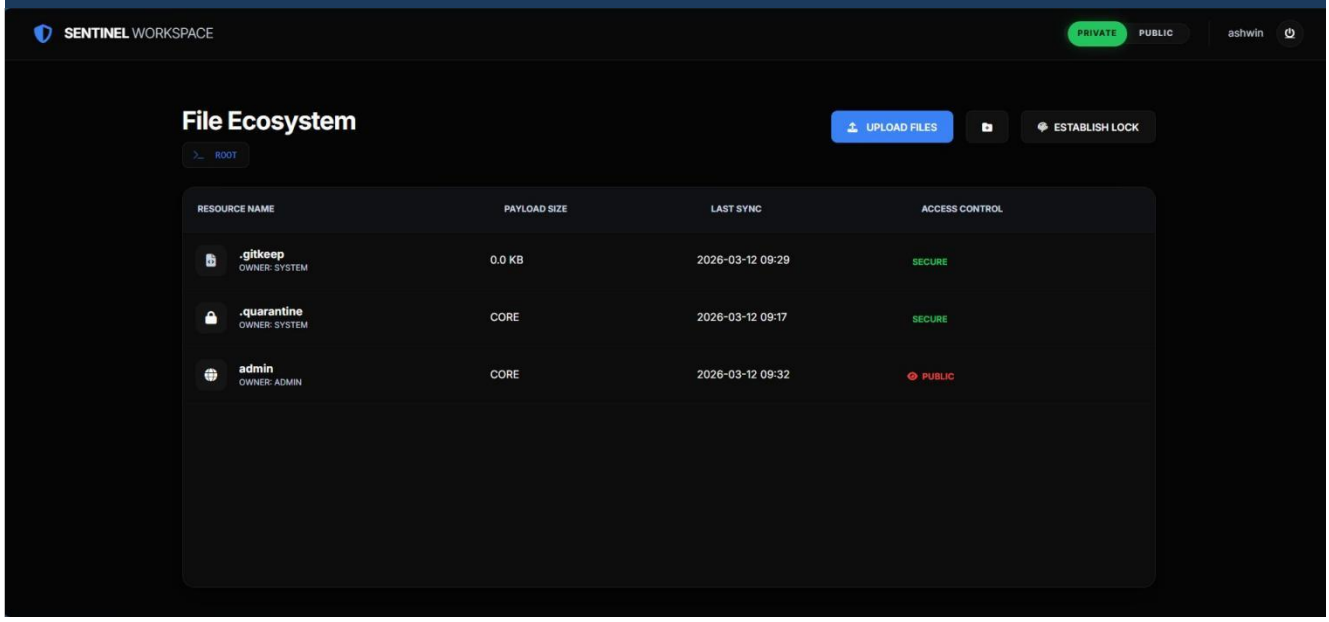

Fig 8 – Sentinel Workspace: Standard user file ecosystem with resource metadata and access control status.

No	UI Element	Description
1	**PRIVATE / PUBLIC toggle**	Switches between private (personal) and public (shared) sector views. PRIVATE active (green) in screenshot.
2	**User indicator (top-right)**	Displays the authenticated username (ashwin) confirming session identity.
3	**UPLOAD FILES button**	Uploads files into accessible sectors. All uploads are monitored by the system.
4	**ESTABLISH LOCK button**	Applies a file lock to a selected resource, restricting modifications until released.
5	**RESOURCE NAME column**	File or sector name with owner tag (OWNER: SYSTEM or OWNER: ADMIN).
6	**PAYLOAD SIZE column**	File size in KB, or 'CORE' for system-managed sectors.
7	**LAST SYNC column**	Timestamp of the most recent modification or sync event for each resource.
8	**ACCESS CONTROL column**	SECURE (green) = private/locked. PUBLIC (red eye icon) = accessible by all users.
9	**.quarantine sector (lock icon)**	System-protected zone. SECURE. Only system/admins may interact with its contents.
10	**admin sector (globe icon)**	Admin-owned sector marked PUBLIC — visible to standard users in this session.

5. Security Features

5.1 Filesystem Event Types

Event	Description
CREATE	New file or directory created in the monitored volume.
MODIFY	Existing file content changed. Triggers content analysis.
RENAME	File or directory renamed. Common ransomware indicator.
DIRECTORY	New subdirectory created — may indicate staging for exfiltration.
OVERWRITE	File content replaced. Monitored for bulk overwrite patterns.

5.2 Severity Classification

Level	Trigger
LOW	Single anomalous event, low confidence. No behavioral patterns matched.
MEDIUM	Multiple weak signals correlated. Suspicious but unconfirmed.
HIGH	Strong behavioral match to known attack patterns. Immediate review required.
CRITICAL	Ransomware-class behavior, bulk exfiltration, or confirmed malware signature.

5.3 Ransomware Detection

- Bulk file encryption: rapid sequential MODIFY/OVERWRITE events across many files within milliseconds.
- Mass rename operations: simultaneous renaming to unfamiliar extensions.
- Entropy increase: files whose content randomness increases sharply, indicating encryption.

5.4 Role-Based Access Control (RBAC)

Role	Permissions
ADMIN	Full access: Live Monitor, Audit Reports, System Files, User Accounts, Log Purge, Sec-Ops mode.
USER	Workspace only: Private/Public file ecosystem. No access to monitoring, user management, or purge functions.

6. Configuration Reference

Variable	Default	Description
IP_DETECTION_MODE	auto	IP address detection mode for server binding.
IP_VERSION_PRIORITY	ipv4	Preferred IP version when multiple addresses exist.
AI_REPORT_API_KEY	(none)	API key for AI-powered report generation. Leave blank to disable.

6.1 Security Hardening

- Change default admin password immediately after deployment.
- Add .env to .gitignore — never commit it.
- Deploy behind Nginx/Apache with HTTPS (TLS 1.2+).
- Restrict dashboard port 5000 via firewall to authorized IP ranges only.
- Set FLASK_ENV=production to disable Flask debug mode.

7. Logging & Reporting

7.1 Log Types

- JSON telemetry logs: machine-readable event records (timestamp, event type, file path, severity, operator). Suitable for SIEM ingestion.
- Text audit logs: human-readable summaries for administrator review and compliance documentation.

Logs are organized by date under logs/ (e.g., logs/mar/ for March).

7.2 Report Generation

generate_report.py compiles log data into structured reports including summary statistics, threat timelines, severity breakdowns, and remediation notes. When an AI API key is configured, AI-generated threat narratives are included. Reports are generated by event index range or manual selection (Section 4.7).

8. Troubleshooting

Problem	Solution
App fails to start	Run pip install -r requirements.txt. Confirm .env exists with all required variables.
Dashboard not accessible	Check Flask started (console output). Verify firewall allows port 5000.
No events in log table	Ensure storage/ directory exists and place files in it. Check monitor.py started without errors.
Login failing	Verify credentials. If locked out, check database_manager.py reset procedures.
AI reports not generating	Confirm AI_REPORT_API_KEY in .env is valid. Check API quota and network connectivity.
High false-positive rate	Adjust heuristic thresholds in config/. Tune threat_engine.py sensitivity to match normal file operation patterns.

9. Contributing

9.1 Branching Strategy

- Feature branches from main: feature/your-feature-name
- Bug fix branches: fix/issue-description
- All changes via pull request. No direct commits to main.
-

9.2 Code Standards

- Maintain Python type hinting in all core/ modules.
- Follow PEP 8 style guidelines.
- All new features must include tests in scripts/.

9.3 Issue Reporting

Report bugs at github.com/Cybernextin/FTMS-Sentinal/issues. Include: description, reproduction steps, expected vs actual behavior, OS, and Python version.

10. Appendix

10.1 Technology Stack

Component	Technology
Backend	Python / Flask (WSGI)
Frontend	HTML5, CSS3, Vanilla JavaScript
File Monitoring	Watchdog – Python filesystem events library
Threat Analysis	Custom heuristic engine (threat_engine.py)
Reporting	Custom Python module with optional AI API integration
Dependency Mgmt	pip + requirements.txt

10.2 Language Distribution

Language	Share
Python	40.3%
HTML	37.5%
JavaScript	20.0%
CSS	2.2%